

TF2026 presentation

Made by **Claudio Marchini**
and **Cesare Tomasi**

Cargoservice: requirements summary

Goal: *cargoservice* automates loading containers into a ship's hold using *cargorobot*.

Layout: 4 storage slots + 1 staging slot (slot5, for barcode marking) + IOPort (button + display) + sonar sensor.

Robot task: move container **IOPort** → **slot5 (marking)**
→ **assigned slot**

Cargoservice: requirements summary

Request flow (button press):

- Occupied IOPort or "out of service" → *retrylater*
- Hold full → reject
- Else → accept, assign a free slot, go **engaged** (Led blinks), 30s to place container or system disengages

Cargoservice: requirements summary

Display always shows: hold status + "Service working" or "Out of service" (sonar D > DFREE for $\geq 3s$ = fault)

Sprint 0: brief outline

The goal of this sprint was to analyze the requirements given to us and to translate them from natural language to a machine-readable representation. The following tasks were completed:

- the *hold*, *slot* and *position* entities were identified and were given matching Java interfaces (**ISlot**, **IHold**, **IPosition**)
- the entire system was identified as *distributed* and *heterogeneous*, which meant implementing a distributed model based on services

Sprint 0: brief outline

- the load request interaction was modeled as *request/reply*
- the already developed **RobotSmart26** was chosen as the cargorobot's implementation (thanks to it being a service and developed in qak, its pathfinding algorithm and its grid based movement)
- the **IOPort** component (used by the client to interact with the system) was also defined as a web GUI

Sprint 1: brief outline

This sprint was built on top of the requirement analysis of **sprint 0** and aimed to build several cardinal components of the system, in particular:

- The main **cargoservice** actor
- The **sonar** subsystem
- The **IOPort** and **pushbutton** interfaces
- Concrete Java implementations and tests of the following interfaces: **ISlot**, **IHold**, **IPosition**

Sprint 1: brief outline

In more detail:

- the sonar hw module was connected to a *Raspberry Pi Pico W* and a python script for communication with both it and the cargoservice actor via *MQTT* (chosen for modularity and lightweightness) was developed
- the IOPort web GUI was finalized (**WebSocket** was chosen over a HTTP request-response style interaction because the component has to maintain the current status of the system at any moment)

Sprint 1: brief outline

- the message format used was defined following the protobook documentation and the actor **sonarwrapper** was introduced to to have a single point where the distance is parsed (making it more robust and easily adjustable in the future)
- the current version of the *cargoservice* actor was now capable of communicating with both working subsystems
- a *Dockerfile* to easily deploy the various components on different nodes was also developed

Sprint 2: brief outline

This sprint was built on top of the work of **sprint 1** and aimed to integrate the last component of the system: the *cargorobot*.

In more detail:

- the *cargoservice* actor was updated and it is now capable to instruct the **RobotSmart26** (taken as is) using requests and dispatches (via TCP)
- the *aril* language (from the docs) was used to model the basic commands of the robot's movement

Sprint 2: brief outline

- the system is now entirely functional and can be tested by the user via a virtual environment containing a simulation of the robot and the hold (available as a **web page**)
- the Dockerfile was updated to handle both the *cargorobot* and the virtual environment

Final system architecture

